Capture the Flag 6: Active or Inactive?

Goal: Make your Student Directory respond to student data by conditionally showing, hiding, and changing UI elements based on each student's state.

Instructor Note

These activities require your own problem-solving and research skills.
DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- Explain an error (not fix your error)
- Clarify documentation

You may **NOT** ask AI to generate the solution/code for a flag.

🚩 Flag 0 — Create Your Feature Branch

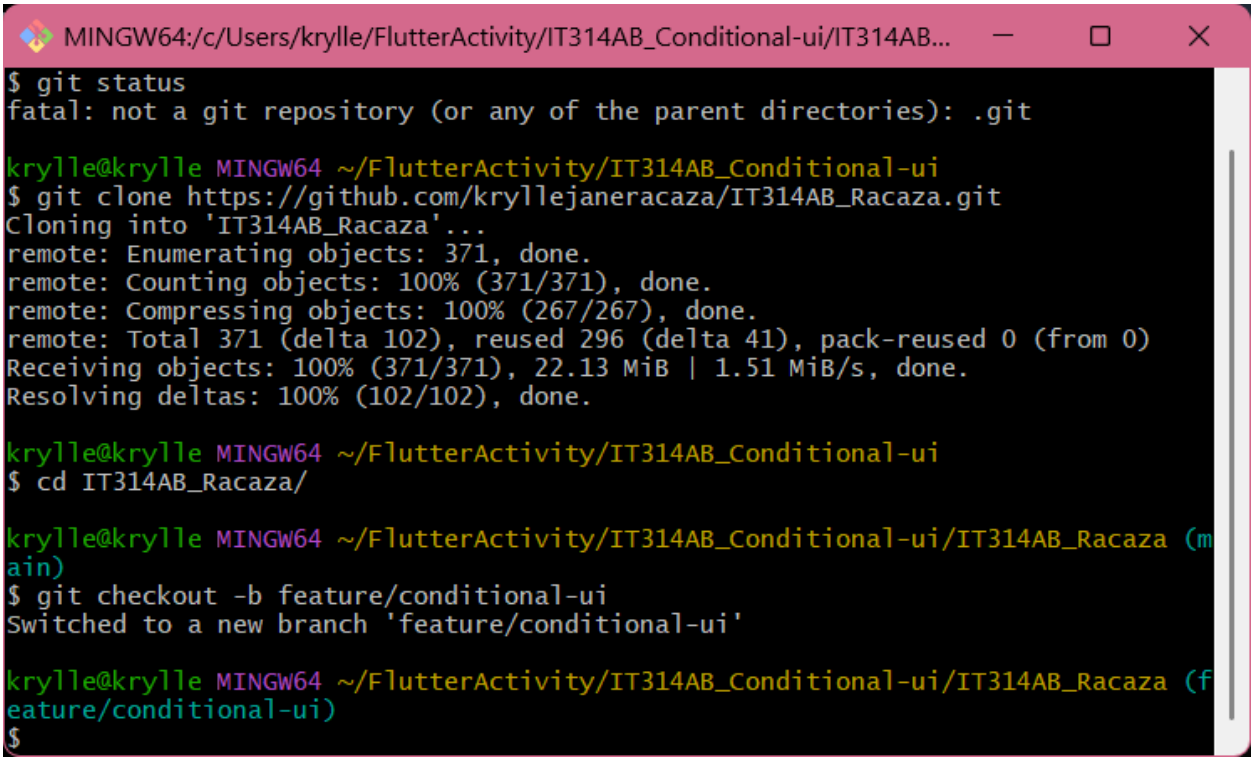
Objective: Create a new feature branch before adding conditional UI.

Create:

`feature/conditional-ui`

Screenshot:

- Terminal showing your current branch



🚩 Flag 1 — Show Active or Inactive

Objective: Change the Student Card based on whether a student is active or inactive.

Add an `active` status to your student data.

Research how Flutter uses **conditional expressions** to display different widgets.

Example states:

- Active
- Inactive

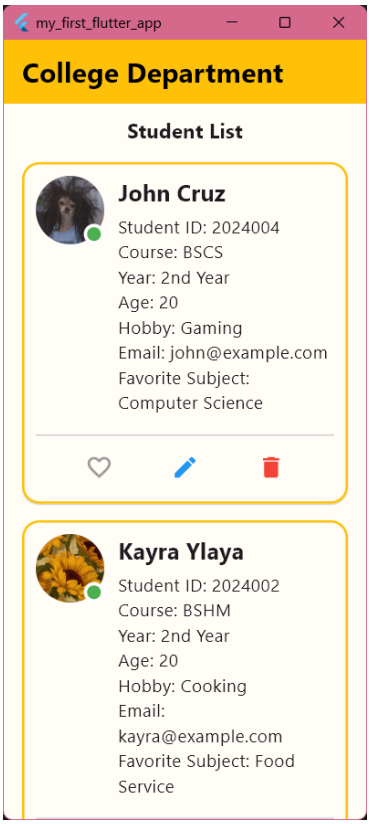
Test

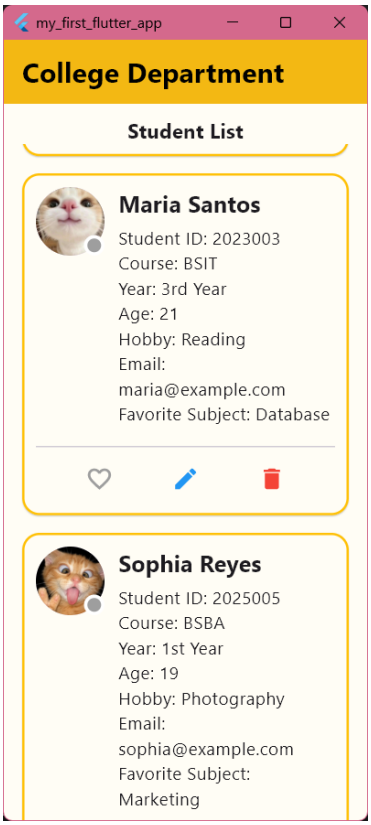
Active students should display an Active indicator.

Inactive students should display an Inactive indicator.

Screenshot:

- Active Student Card
- Inactive Student Card
- Conditional logic in your code






```
      child: Tooltip(
        message: student['active'] ? 'Active' : 'Inactive',
        child: Stack(
          children: [
            CircleAvatar(
              radius: 30,
              backgroundImage: AssetImage(student['image']),
            ), // CircleAvatar
            Positioned(
              right: 0,
              bottom: 0,
              child: Container(
                width: 18,
                height: 18,
                decoration: BoxDecoration(
                  color: student['active'] ? Colors.green : Colors.grey,
                  shape: BoxShape.circle,
                  border: Border.all(color: Colors.white, width: 3),
                ), // BoxDecoration
              ), // Container
            ), // Positioned
          ],
        ), // Stack
      ), // Tooltip
    ); // GestureDetector
  }
```

🚩 Flag 2 — Show or Hide

Objective: Display certain UI elements only when a condition is true.

Add a conditional element to the Student Card.

Examples:

- Favorite badge appears only when favorited
- Warning appears only when inactive
- Delete action appears only if active

Research ways to conditionally include or exclude widgets from the widget tree.

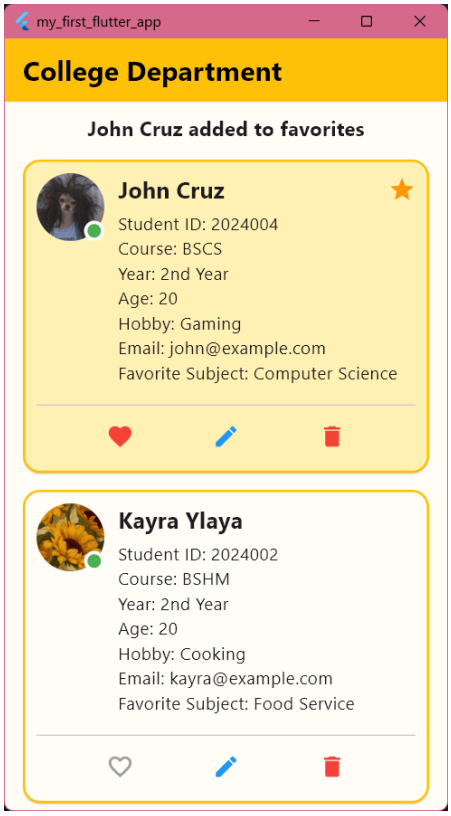
Test

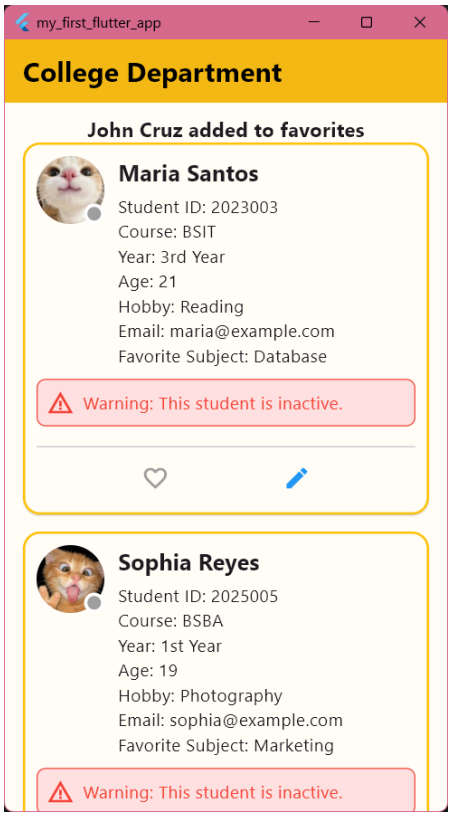
When the condition is true, the element appears.

When the condition is false, the element disappears.

Screenshot:

- UI with the element visible
- UI with the element hidden
- Conditional widget code






```
            if (student['favorite'])
              const Icon(
                Icons.star,
                color: Colors.orange,
                size: 24,
              ), // Icon
          ],
        ), // Row
        const SizedBox(height: 5),
        Text(
          'Student ID: ${student['studentId']}\n'
          'Course: ${student['course']}\n'
          'Year: ${student['year']}\n'
          'Age: ${student['age']}\n'
          'Hobby: ${student['hobby']}\n'
          'Email: ${student['email']}\n'
          'Favorite Subject: ${student['favoriteSubject']}',
        ), // Text
      ],
    ), // Column
  ), // Expanded
], // Row
```

```
if (!student['active'])
  Container(
    width: double.infinity,
    margin: const EdgeInsets.only(top: 10),
    padding: const EdgeInsets.all(8),
    decoration: BoxDecoration(
      color: const Color(0xFFFFE0E0),
      borderRadius: BorderRadius.circular(8),
      border: Border.all(color: Colors.red),
    ), // BoxDecoration
    child: const Row(
      children: [
        Icon(Icons.warning_amber, color: Colors.red),
        SizedBox(width: 8),
        Expanded(
          child: Text(
            'Warning: This student is inactive.',
            style: TextStyle(color: Colors.red),
          ), // Text
        ), // Expanded
      ],
    ), // Row
  ), // Container
```

```
const SizedBox(height: 10),
const Divider(),
Row(
  mainAxisAlignment: MainAxisAlignment.spaceEvenly,
  children: [
    IconButton(
      icon: Icon(
        student['favorite']
            ? Icons.favorite
            : Icons.favorite_border,
        color: student['favorite'] ? Colors.red : Colors.grey,
      ), // Icon
      onPressed: () {
        setState(() {
          student['favorite'] = !student['favorite'];
          message = student['favorite']
              ? '${student['name']} added to favorites'
              : '${student['name']} removed from favorites';
        });
      },
    ), // IconButton
    IconButton(
      icon: const Icon(Icons.edit, color: Colors.blue),
      onPressed: () {
        showDialog(
          context: context,
          builder: (context) {
            return AlertDialog(
              title: const Text(
                'Edit Student',
                style: TextStyle(fontWeight: FontWeight.bold),
              ), // Text
```

```
              ), // Text
              content: Text(
                'You selected ${student['name']} for editing.',
              ), // Text
              actions: [
                TextButton(
                  onPressed: () => Navigator.pop(context),
                  child: const Text('Close'),
                ), // TextButton
              ],
            ); // AlertDialog
          },
        );
      },
    ), // IconButton

    if (student['active'])
      IconButton(
        icon: const Icon(Icons.delete, color: Colors.red),
        onPressed: () {
          setState(() {
            students.remove(student);
            message = '${student['name']} deleted';
          });
        },
      ), // IconButton
  ],
), // Row
],
), // Column
), // Padding
), // Card
```

🏴 Flag 3 — The Empty Directory

Objective: Handle situations where there are no students to display.

Modify your Student Directory so it detects when the student list is empty.

Create an **empty state**.

Examples:

- "No students found"
- Empty-state icon
- Button to add a student

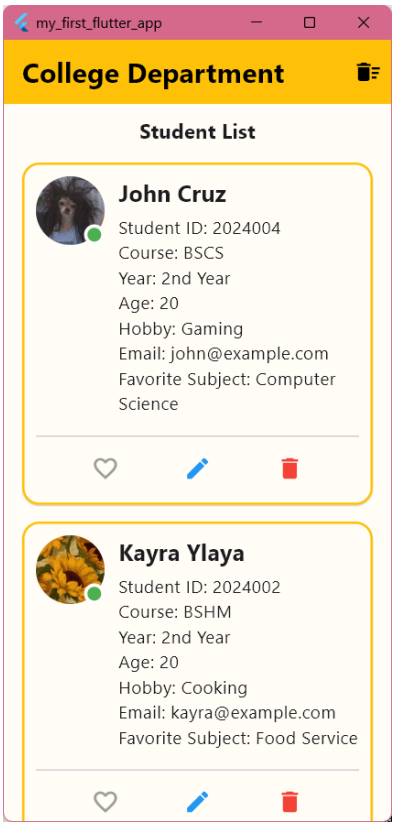
Test

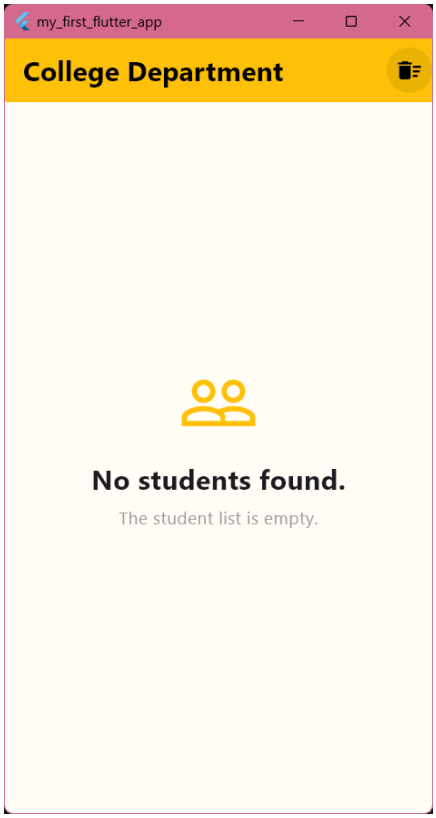
Delete all students in your application (not the code)

Instead of showing a blank screen, your application should display your empty state.

Screenshot:

- Empty student list
- Empty-state UI
- Condition checking for an empty list






```
body: students.isEmpty
    ? Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Icon(
              Icons.people_outline,
              size: 70,
              color: Color(0xFFFFC107),
            ), // Icon
            const SizedBox(height: 15),
            const Text(
              'No students found.',
              style: TextStyle(fontSize: 22, fontWeight: FontWeight.bold)
            ), // Text
            const SizedBox(height: 5),
            const Text(
              'The student list is empty.',
              style: TextStyle(color: Colors.grey),
            ), // Text
            const SizedBox(height: 20),
          ],
        ), // Column
      ) // Center
    : Column(
        children: [
          const SizedBox(height: 10),
          Text(
            message,
```

🚩 Flag 4 — Loading the Directory

Objective: Display different UI while student data is being loaded.

Create a loading state for your Student Directory.

Research how Flutter can conditionally display a loading indicator.

Examples:

- `CircularProgressIndicator`
- Loading message
- Loading placeholder

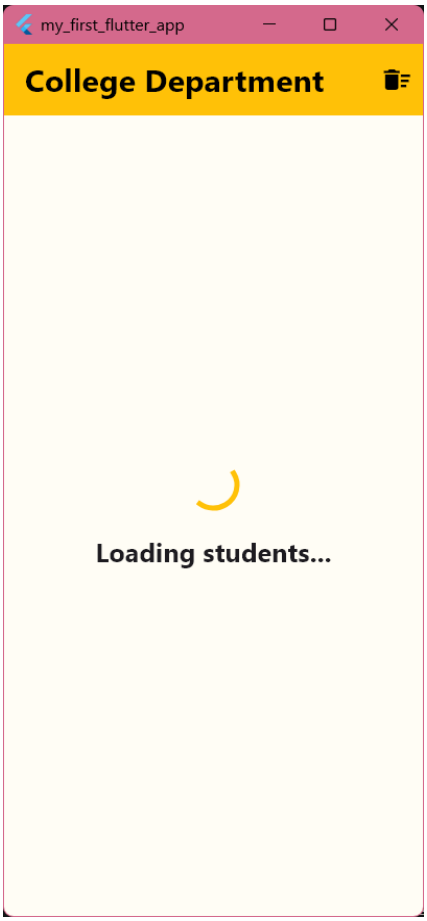
Test

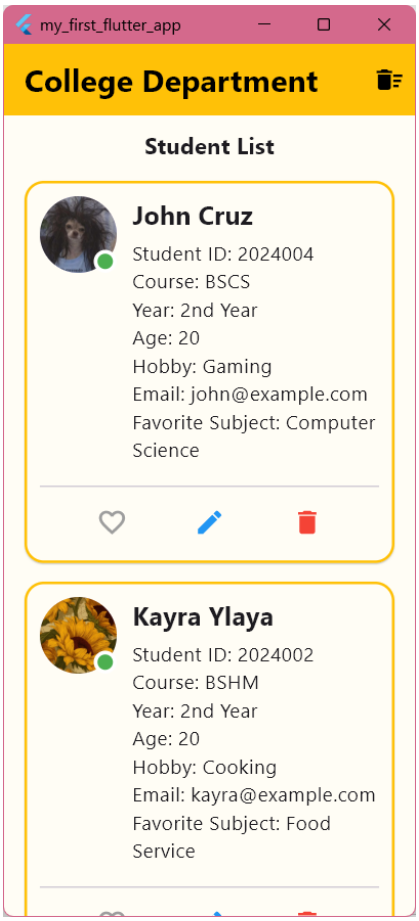
When the application is in a loading state, the student list should not be displayed as the main content.

Once loading is complete, the Student Directory should appear.

Screenshot:

- Loading state
- Loaded state
- Conditional loading logic






```dart
class _StudentListState extends State<StudentList> {
  String message = 'Student List';
  bool isLoading = true;

  @override
  void initState() {
    super.initState();
    loadStudents();
  }

  Future<void> loadStudents() async {
    setState(() {
      isLoading = true;
    });

    await Future.delayed(const Duration(seconds: 10));

    if (!mounted) return;

    setState(() {
      isLoading = false;
      message = 'Student List';
    });
  }

  void deleteAllStudents() {
    setState(() {
      students.clear();
    });
  }
```

```dart
body: isLoading
    ? const Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            CircularProgressIndicator(color: Color(0xFFFFC107)),
            SizedBox(height: 20),
            Text(
              'Loading students...',
              style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
            ), // Text
          ],
        ), // Column
      ) // Center
```

🚩 Flag 5 — Multiple States

Objective: Make your Student Directory handle multiple UI states correctly.

Your application should now be able to display:

1. Loading
2. Students available
3. No students available

Determine which state should be displayed based on your data.

Test

Verify each state:

Loading
→ Loading UI

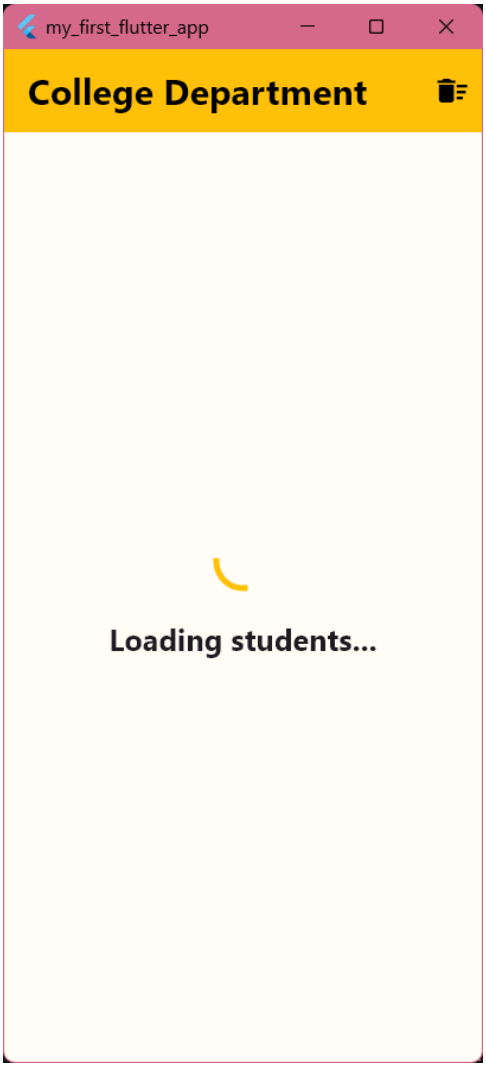
Students available
→ Student List

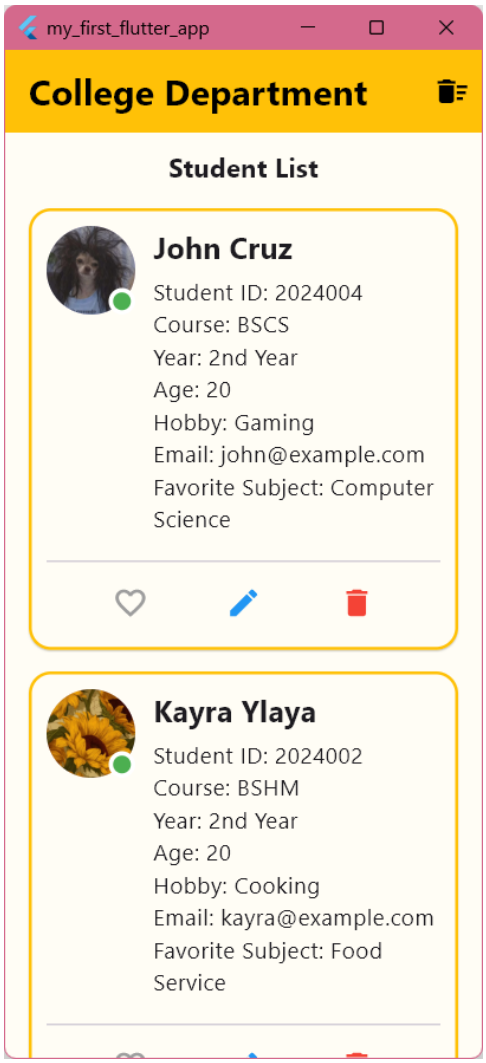
Empty list
→ Empty State

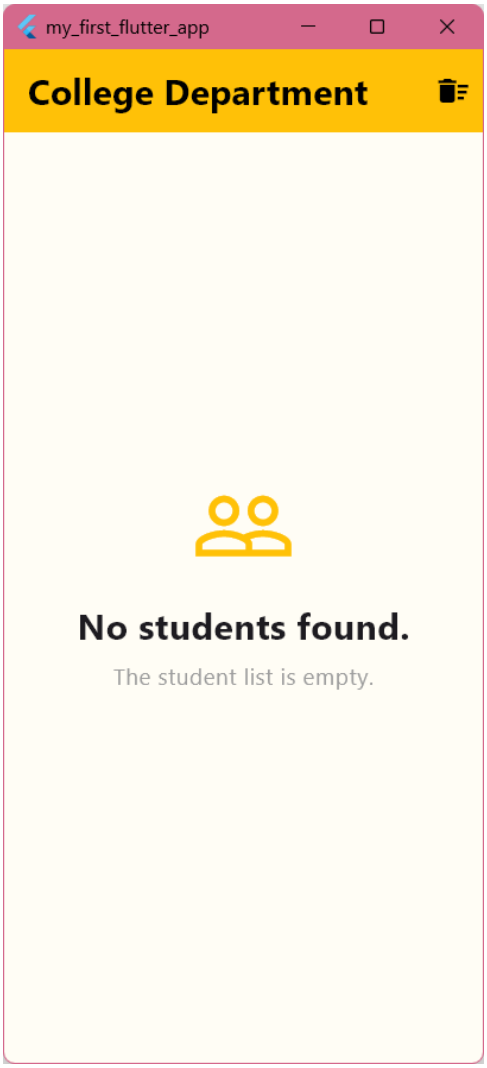
Only the appropriate UI should appear at a time.

Screenshot:

- Loading state
- Student list state
- Empty state


my_first_flutter_app
College Department
Loading students...


my_first_flutter_app
College Department
Student List
John Cruz
Student ID: 2024004
Course: BSCS
Year: 2nd Year
Age: 20
Hobby: Gaming
Email: john@example.com
Favorite Subject: Computer Science
Kayra Ylaya
Student ID: 2024002
Course: BSHM
Year: 2nd Year
Age: 20
Hobby: Cooking
Email: kayra@example.com
Favorite Subject: Food Service


my_first_flutter_app
College Department
No students found.
The student list is empty.

🏴 Flag 6 — States Work Together

Objective: Make sure conditional UI does not break your existing interactions.

Test the combination of:

- Favorite
- Delete
- Edit
- Active/Inactive
- Show/Hide
- Empty state
- Loading state

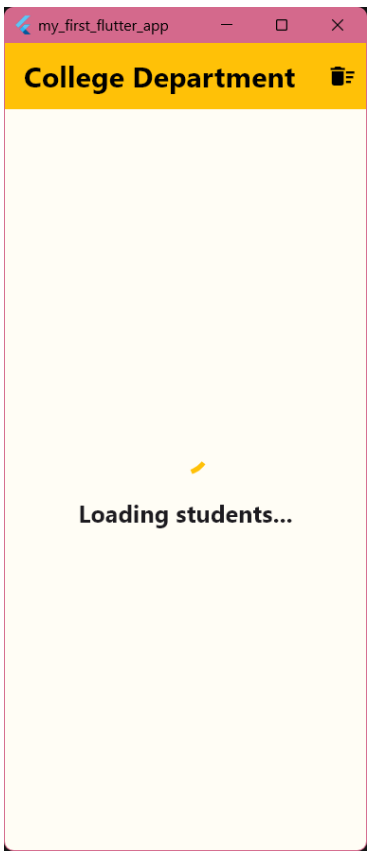
Test Scenario

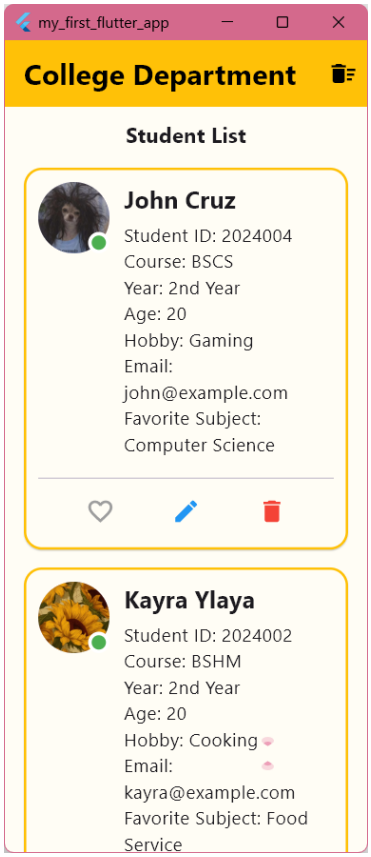
1. Favorite a student.
2. Change or display their Active/Inactive state.
3. Delete students until the directory becomes empty.
4. Verify the Empty State appears.
5. Reload or restore students.
6. Verify the Student Directory appears again.

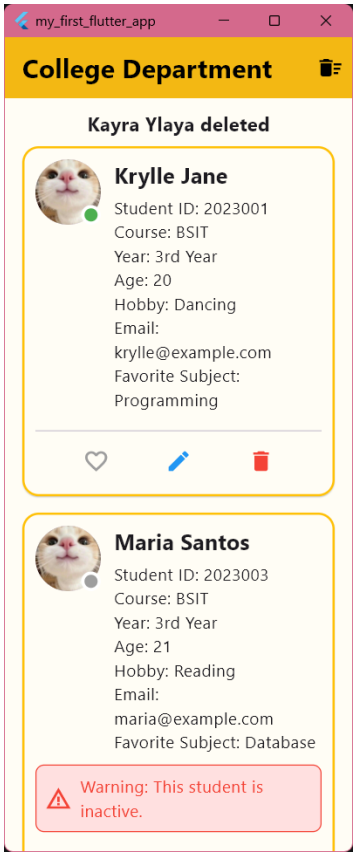
The application should continue functioning correctly as its state changes.

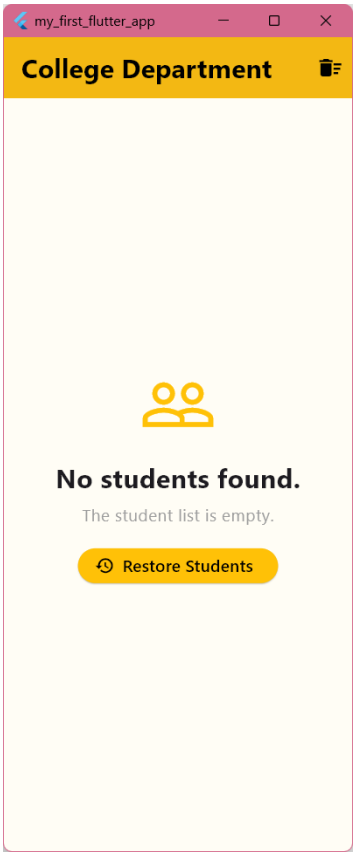
Screenshot:

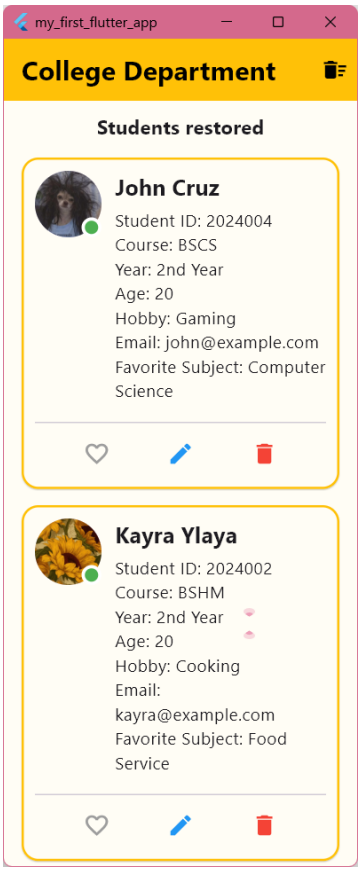
- Multiple conditional states working
- Interactive Student Card
- Empty State
- Application functioning normally

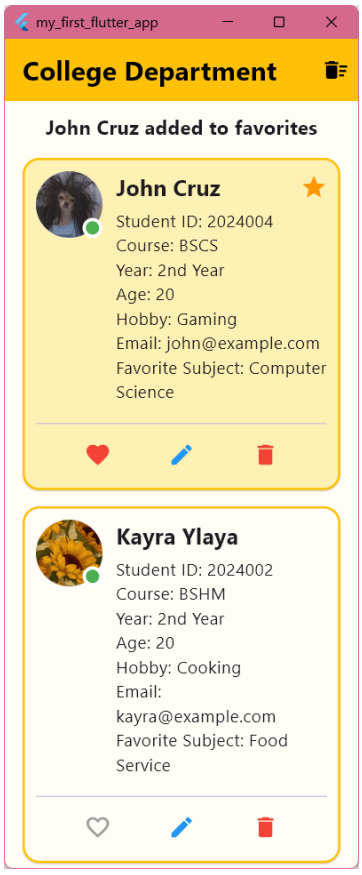













```
void restoreStudents() { //nakalimtan ang restore HAHHAAHHA
  setState(() {
    students = defaultStudents();
    students.sort((a, b) => a['name'].compareTo(b['name']));
    message = 'Students restored';
  });
}
```

```
ElevatedButton.icon(
  onPressed: restoreStudents,
  icon: const Icon(Icons.restore),
  label: const Text('Restore Students'),
  style: ElevatedButton.styleFrom(
  backgroundColor: const Color(0xFFFFC107),
  foregroundColor: Colors.black,
  ),
) // ElevatedButton.icon
```

🏴 Flag 7 — Save Your Progress

Objective: Commit and push your completed Conditional UI Student Directory.

Use a proper Git commit message.

Branch:

`feature/conditional-ui`

Screenshot:

- Successful commit
- Current branch
- Terminal showing the commit

